

MODULE 4 · NETWORKING

CoreDNS & Service Discovery

Reaching Pods and Services by name, not by IP

Names outlive IPs — hardcode a name, not an address

Pod IPs churn on every reschedule, and even a Service's ClusterIP is only stable for that Service's lifetime. Nothing should ever hardcode an IP.

CoreDNS is the cluster's built-in name server — it lets every Pod resolve a Service by name and get back whatever ClusterIP is currently correct.

BY THE END YOU CAN

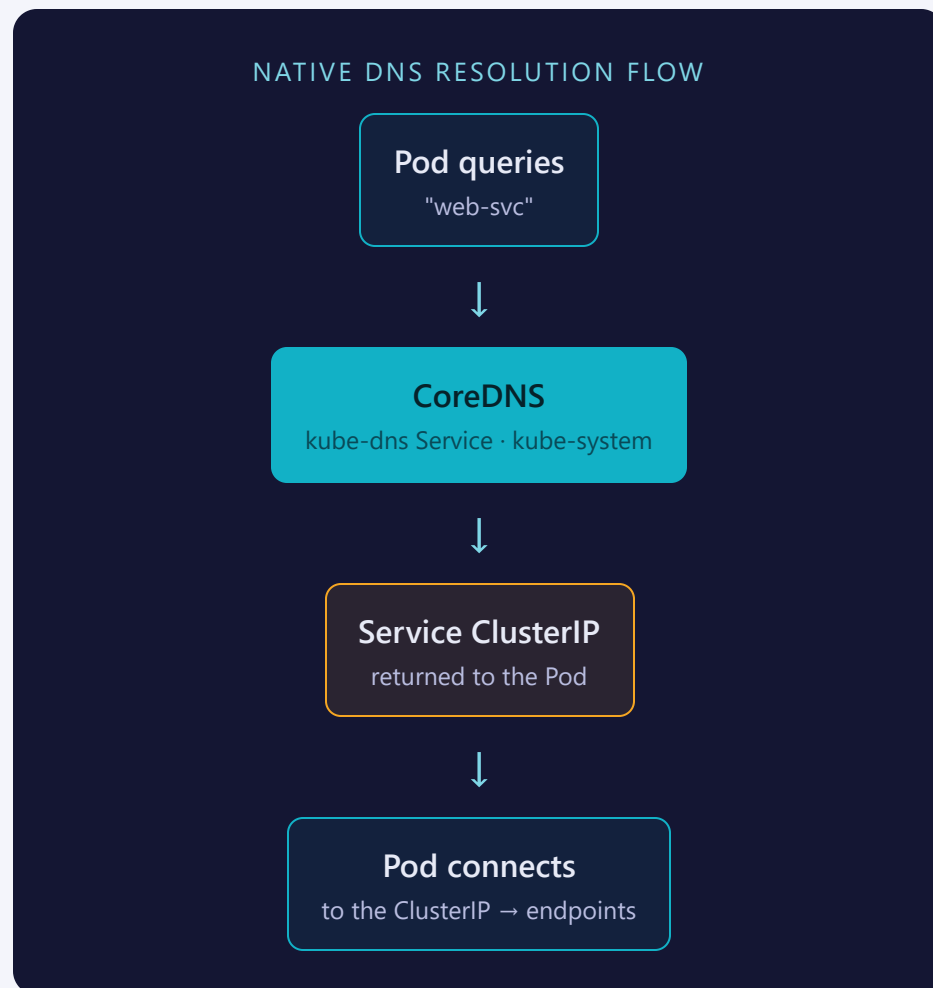
- Explain what CoreDNS is and where it runs
- Read the FQDN `svc.ns.svc.cluster.local`
- Use short names and know when they work
- Read a Pod's `/etc/resolv.conf`

CONCEPT

CoreDNS is the cluster's DNS server

CoreDNS runs as an ordinary **Deployment in kube-system**, fronted by the kube-dns Service — a ClusterIP whose name is historical (CoreDNS replaced kube-dns but kept its address).

The kubelet injects that fixed ClusterIP into every Pod's `/etc/resolv.conf` as its **nameserver**. So a name lookup always has somewhere to go.



`svc.ns.svc.cluster.local`, piece by piece

SEGMENT	EXAMPLE	MEANING
<code><service></code>	<code>web-svc</code>	the Service's name
<code><namespace></code>	<code>training</code>	where that Service lives
<code>svc</code>	<code>svc</code>	fixed — "this is a Service", not a Pod
<code>cluster.local</code>	<code>cluster.local</code>	the cluster's DNS zone (the default)

SAME-NAMESPACE SHORTCUT

A Pod can drop everything after the Service name if it's calling within its own namespace — the resolver's search list fills in the rest.

What the kubelet writes into every Pod

```
nameserver 10.96.0.10      # CoreDNS ClusterIP (kube-dns Svc)
search training.svc.cluster.local # tried, in order, ...
      svc.cluster.local cluster.local # ...until one resolves
options ndots:5           # <5 dots → try search list first
```

- **nameserver** — always the CoreDNS ClusterIP, never a Pod IP
- **search** — namespace-scoped suffixes tried before giving up
- **ndots** — how many dots a name needs before it's tried as-is

WATCH

ndots:5 means most external names try every search suffix first. A trailing dot (example.com.) skips the search list.

Short name in-namespace, full name across it

SAME NAMESPACE — SHORT NAME

```
wget -q0- http://web-svc
```

ANY NAMESPACE — FQDN

```
wget -q0- http://web-svc.training.svc.cluster.local
```

WHY THE SHORT NAME WORKS

The client's own namespace seeds the search list, so `web-svc` expands to `web-svc.training.svc.cluster.local` automatically. Cross-namespace calls skip that luck and must spell out at least `<service>.<namespace>`.

SEE IT

A resolved name proves the wiring

nslookup shows two things: which server answered (always CoreDNS) and the address it handed back. If that address matches the Service's ClusterIP, DNS is doing its job.

No answer at all points at CoreDNS itself or a NetworkPolicy blocking port 53 — not the Service.

```
$ kubectl exec test-client -n training -- \
    nslookup web-svc.training.svc.cluster.local
Server:    10.96.0.10
Address:   10.96.0.10:53

Name:      web-svc.training.svc.cluster.local
Address: 10.96.44.17
```

PUNCHLINE

A 404 can be the success signal

To prove DNS truly crosses namespace boundaries, look up a Service that lives somewhere else entirely — then follow through with an actual HTTP call, not just a lookup.

A resolved name **plus** a response — even an error response — proves the full path: DNS resolution, then routing, then a workload actually answering.

```
$ kubectl exec test-client -n training -- \
  wget -q0- --timeout=5 \
  http://ingress-nginx-controller.ingress-
  nginx.svc.cluster.local
HTTP/1.1 404 Not Found
```

NOT A FAILURE

The name resolved and a Pod answered — the 404 is an application-layer detail, not a DNS problem.

Where people trip

- **Bare name, wrong namespace.** Short names only resolve in the Pod's own namespace — cross-namespace needs <svc>. <ns> at minimum.
- **CoreDNS down = cluster-wide outage.** Every name-based call fails at once — check kube-system pods before anything else.
- **ndots:5 slows external lookups.** Non-cluster names try every search suffix before the real one resolves.
- **NetworkPolicy blocks DNS silently.** A default-deny egress policy that misses port 53 (UDP *and* TCP) breaks every lookup with no obvious error.

Commands to keep at hand

```
$ kubectl -n kube-system get pods,svc -l k8s-app=kube-dns # CoreDNS + kube-dns Svc
$ kubectl exec POD -- nslookup NAME # resolve + who answered
$ kubectl exec POD -- cat /etc/resolv.conf # nameserver, search, ndots
$ kubectl -n kube-system get configmap coredns -o yaml # the Corefile
$ kubectl exec POD -- wget -qO- http://SVC.NS.svc.cluster.local # round-trip test
```

Tip: resolve first, then connect. It tells you in one step whether a broken call is a DNS problem or something further downstream.

RECAP

CoreDNS in one breath

- CoreDNS = cluster DNS — a Deployment in kube-system behind the kube-dns Service
- Full name: `svc.ns.svc.cluster.local`
- Short names work only inside the calling Pod's own namespace
- search in `resolv.conf` is what makes the short name work

HANDS-ON

Now run Lab 4.2
[labs/4.2/](#)

Next: [4.3 — Pod-to-Pod Connectivity](#)